CATEGORY API

Admin — Add Product Category & Get API

Complete build guide: Middleware → Model → Controller → Routes → Testing

> **How to use this document**
> Follow the steps in order: STEP 1 → STEP 4. Each step explains WHAT the code does and WHY it's written that way — not just copy-paste. A dedicated section on async/await is included near the end, since that's what most people get stuck on.

STEP 1 — Authentication Middleware (middleware/auth.js)

Middleware is a function that runs BEFORE your route handler. It checks something about the request — here, whether the user is logged in — and either lets the request continue or blocks it.

What this file does

It acts as a gatekeeper: before any protected route (like creating, updating, or deleting a category) can be reached, this middleware checks whether the request is coming from a logged-in, valid user.

Step-by-step logic

1. Extract the token — reads the Authorization header from the incoming request, where the client sends it as: Bearer <token>.
2. Verify the token — jwt.verify() checks the token against your secret key (process.env.JWT_SECRET) to confirm it is genuine, hasn't expired, and hasn't been tampered with.
3. Attach user data — once verified, the decoded user info (id, role) is attached to req.user, so later code (controllers) knows exactly who is making the request.
4. Call next() — if everything checks out, control passes to the next middleware or the actual route handler.
5. Error handling — if the token is missing, malformed, or expired, the middleware immediately responds with 401 Unauthorized and the request goes no further.

Final code — middleware/auth.js

```
const jwt = require('jsonwebtoken');

// PROTECT – checks the user is logged in (valid token)
exports.protect = (req, res, next) => {
  try {
    const authHeader = req.headers['authorization'];
```

```
    if (!authHeader) {
      return res.status(401).json({ success: false, message: 'No token, authorization denied' });
    }

    const token = authHeader.split(' ')[1]; // "Bearer TOKEN"

    if (!token) {
      return res.status(401).json({ success: false, message: 'Invalid token format' });
    }

    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = decoded; // { id, role }

    next();
  } catch (err) {
    return res.status(401).json({ success: false, message: 'Token invalid or expired' });
  }
};

// ADMIN ONLY — must run AFTER protect, relies on req.user
exports.adminOnly = (req, res, next) => {
  if (!req.user || req.user.role !== 'admin') {
    return res.status(403).json({ success: false, message: 'Admin access only' });
  }
  next();
};
```

Important fix
This file must live at middleware/auth.js (spelled correctly, singular "middleware"). Anywhere you import it, always use the same exact folder name — a mismatch like ../middelwares/auth is the #1 cause of "Cannot find module" crashes.

Setup requirement
Add JWT_SECRET=your_secret_key to your .env file, and load it with require('dotenv').config() at the top of your server file. Without this, jwt.verify() will always fail.

STEP 2 — Category Model (models/Category.js)

The model defines the SHAPE of your data — what fields a category document has in MongoDB, and what rules those fields must follow before saving.

Required fields

- name — the category's display name (e.g. "Electronics"). Required and unique — no two categories can share a name.
- description — optional text describing the category.
- slug — a URL-friendly version of the name (e.g. "home-appliances"). Must be unique and lowercase.
- createdBy — a reference to the User who created this category (stores that user's MongoDB _id).

Why slug exists alongside name

name is for display — it can have capital letters, spaces, and symbols. slug is for use in URLs — clean and safe, e.g. /api/categories/home-appliances instead of an ugly URL with a raw database _id. The schema does not generate the slug by itself — it just stores it and enforces uniqueness/lowercase. The actual value is built in the controller (see Step 3) from the name field.

Why createdBy uses ObjectId + ref

This is how Mongoose creates relationships between collections — similar to a foreign key in SQL. It doesn't copy the user's data into the category document; it just stores a reference (their _id). Later, you can use .populate('createdBy') to fetch the full user details when needed, without duplicating data.

Final code — models/Category.js

```
const mongoose = require('mongoose')

const categorySchema = new mongoose.Schema({
    name: {
        type: String,
        required: true,
        unique: true,
        trim: true
    },
    description: {
        type: String,
    },
    slug: {
        type: String,
        unique: true,
        lowercase: true
    },
    createdBy: {
        type: mongoose.Schema.Types.ObjectId,
        ref: 'User'
    }
}, { timestamps: true })

module.exports = mongoose.model('Category', categorySchema)
```

Why required: true and unique: true matter
These are validation rules enforced by Mongoose BEFORE saving to the database. required stops a category from being saved with no name. unique tells MongoDB to build an index that rejects duplicate values — so two categories can never both be named "Electronics", and two categories can never both end up with the same slug.

STEP 3 — Category Controller (controllers/categoryController.js)

The controller holds the actual business logic — what happens when a request hits a route. Each exported function corresponds to one API action: create, get all, update, delete.

Final code — controllers/categoryController.js

```
const Category = require('../models/Category');

// @desc   Create new category
// @route  POST /api/categories
// @access Admin only
exports.createCategory = async (req, res) => {
  try {
    const { name, description } = req.body;

    const slug = name.toLowerCase().trim().replace(/\s+/g, '-');

    const existing = await Category.findOne({ name });
    if (existing) {
      return res.status(400).json({ success: false, message: 'Category already exists' });
    }

    const category = await Category.create({
      name,
      description,
      slug,
      createdBy: req.user.id
    });

    res.status(201).json({ success: true, data: category });
  } catch (err) {
    res.status(500).json({ success: false, message: err.message });
  }
};

// @desc   Get all categories
// @route  GET /api/categories
// @access Public
exports.getCategories = async (req, res) => {
  try {
    const categories = await Category.find().sort({ createdAt: -1 });
    res.status(200).json({ success: true, count: categories.length, data: categories });
  } catch (err) {
    res.status(500).json({ success: false, message: err.message });
  }
};

// @desc   Update category
// @route  PUT /api/categories/:id
// @access Admin only
exports.updateCategory = async (req, res) => {
  try {
    const category = await Category.findByIdAndUpdate(req.params.id, req.body, {
      new: true,
      runValidators: true
```

```
    });

    if (!category) {
      return res.status(404).json({ success: false, message: 'Category not found' });
    }

    res.status(200).json({ success: true, data: category });
  } catch (err) {
    res.status(500).json({ success: false, message: err.message });
  }
};

// @desc   Delete category
// @route  DELETE /api/categories/:id
// @access Admin only
exports.deleteCategory = async (req, res) => {
  try {
    const category = await Category.findByIdAndDelete(req.params.id);

    if (!category) {
      return res.status(404).json({ success: false, message: 'Category not found' });
    }

    res.status(200).json({ success: true, message: 'Category deleted' });
  } catch (err) {
    res.status(500).json({ success: false, message: err.message });
  }
};
```

★ Why async / await is used in every function

This is the part that trips up most beginners — here's the full reasoning.

1. Database calls are not instant

Category.findOne(), Category.create(), Category.find(), findByIdAndUpdate(), and findByIdAndDelete() all talk to MongoDB over a network connection. That takes time — maybe 5ms, maybe 200ms depending on load. JavaScript does not pause and wait by default; it keeps running the next line immediately. Without something to make it wait, your code would try to send a response (res.status(201).json(...)) before the database had even finished creating the category — resulting in undefined data or a crash.

2. Mongoose functions return a Promise

Every Mongoose method that touches the database (find, create, findByIdAndUpdate, etc.) returns a Promise — an object representing "a value that will exist eventually, either successfully (resolved) or with an error (rejected)". await is what actually pauses your function's execution until that Promise resolves, and then hands you back the real result.

3. What await does, concretely

Look at this specific line from createCategory:

```
const existing = await Category.findOne({ name });
```

Without await, existing would be a pending Promise object — not the actual category (or null). Your very next line, if (existing), would then be checking the wrong thing entirely. With await, JavaScript pauses this function only (not the whole server) until MongoDB replies, and existing becomes the real result: either a category document or null.

4. Why async is required on the function itself

await can only be used inside a function marked async. That's a hard JavaScript rule — it's not optional styling. This is why every controller function here is written as:

```
exports.createCategory = async (req, res) => { ... }
```

Marking the function async also means it always returns a Promise itself, and lets you use try/catch around the awaited calls to handle failures cleanly (see next point).

5. Why try/catch surrounds every await

A Promise can fail — the database might be down, the connection might drop, or invalid data might violate a schema rule (like a duplicate unique name). When an awaited Promise rejects, it throws an error at that exact line. Wrapping the code in try/catch lets you catch that error and send a proper response (res.status(500).json({...})) instead of the entire server crashing with an unhandled exception.

6. Why NOT use .then().catch() chains instead

Mongoose still supports the older .then()/.catch() Promise style, but async/await was chosen here because it reads top-to-bottom like normal synchronous code, avoids deeply nested .then() chains ("callback pyramid"), and pairs naturally with try/catch for error handling — which is exactly the pattern every function above follows.

> **In one sentence**
> async/await exists so your code can wait for the database to actually respond before using that data — without it, every controller here would send back empty or broken responses, or crash outright.

STEP 4 — Category Routes (routes/categoryRoutes.js)

Routes connect a URL + HTTP method to the correct controller function, and decide which middleware must run first.

Final code — routes/categoryRoutes.js

```
const router = require('express').Router();
const {
  createCategory,
  getCategories,
  updateCategory,
  deleteCategory
} = require('../controllers/categoryController');
const { protect, adminOnly } = require('../middleware/auth'); // NOT "middelwares"

router.get('/', getCategories);                          // public — sab dekh sakte hain
router.post('/', protect, adminOnly, createCategory);     // admin only
router.put('/:id', protect, adminOnly, updateCategory);   // admin only
router.delete('/:id', protect, adminOnly, deleteCategory);// admin only

module.exports = router;
```

Why the middleware order matters

protect runs first and attaches req.user by decoding the token. adminOnly runs second and reads req.user.role — it depends entirely on protect having already run. If you reversed the order, adminOnly would crash trying to read .role off of undefined.

Mounting the routes in your server file

```
app.use('/api/categories', require('./routes/categoryRoutes'));
```

This line must come after express.json() (so req.body is parsed) and after your MongoDB connection is established.

Testing Checklist (Postman)

Follow this order every time you test — skipping steps is the most common source of confusing 401/403 errors.

1. Get a token first — log in with an admin user's email/password via your login endpoint. Copy the token from the response.
2. Test the public route — GET /api/categories with no headers. Should return 200 with an array (empty at first).
3. Test the protected route — POST /api/categories with header Authorization: Bearer <token> and body { "name": "Electronics", "description": "Gadgets" }. Should return 201.
4. Test adminOnly enforcement — repeat step 3 using a token from a NON-admin user. Should return 403 Admin access only.
5. Test update — PUT /api/categories/:id using the _id from step 3, with the admin token.
6. Test delete — DELETE /api/categories/:id using the same _id and admin token.

> **Common issues this document already fixes**
> 1) Folder name mismatch (middelwares vs middleware) causing module-not-found crashes. 2) Missing role in the JWT payload — role must be included when you sign the token at login, or adminOnly will always return 403. 3) Forgetting await on a Mongoose call, which silently breaks the response with an unresolved Promise instead of real data.